__init__.py

`src/moe_pruning/models/__init__.py · 139 lines · module moe_pruning.models`

What it does

The model plugin registry. A plugin bundles everything model-specific: an adapter, a task objective and the Hugging Face auto class. The rest of the code looks plugins up by the model's `config.model_type`.

How it works

- `ModelPlugin` describes a plugin; `register_plugin()` adds it and rejects two plugins claiming the same model type.
- `load_plugins()` imports built-in plugins (sub-packages of `models/`), entry points in the `moe_pruning.models` group, and modules listed in `MODEL_PLUGINS`.
- `get_plugin()`, `plugin_for_model()` and `plugin_for_source()` return the plugin for a type, a loaded model, or a model path / Hub id (reading only its `config.json`).

Notes

- The module docstring contains a complete example of adding a new model.

Defined in this file

Extracted from the source code (classes, public functions and methods).

Name	Description
`class ModelPlugin`	Adapter, task and HF auto class of one model family.
`.adapter()`	New adapter instance.
`.task()`	New task-objective instance.
`register_plugin(plugin)`	Add a plugin; reject a second plugin for the same model type.
`load_plugins(extra_modules)`	Import built-in plugins, entry-point plugins and MODEL_PLUGINS modules (once).
`available_plugins()`	All registered plugins by model type.
`get_plugin(model_type)`	Plugin for a model type (clear error if unsupported).
`plugin_for_model(model)`	Plugin of a loaded model.
`plugin_for_source(source, revision, cache_dir)`	Plugin of a local model directory or Hub id (reads only config.json).

Connections

Uses: `moe_pruning.model.architecture`, `moe_pruning.models`, `moe_pruning.tasks.base`, `moe_pruning.tasks.causal_lm`

Used by: `src/moe_pruning/model/architecture.py`, `src/moe_pruning/model/loader.py`, `src/moe_pruning/models/olmoe/__init__.py`, `src/moe_pruning/models/switch_transformers/__init__.py`, `src/moe_pruning/serialization/checkpoint.py`, `src/moe_pruning/utils/cli.py`